

AuthiChain Database Migrations Guide

Version: 1.0.0

Last Updated: October 19, 2025

Database: PostgreSQL with Prisma ORM



Table of Contents

1. [Overview](#)
 2. [Database Schema](#)
 3. [Quick Start](#)
 4. [Migration Commands](#)
 5. [Seeding Data](#)
 6. [Schema Overview](#)
 7. [Troubleshooting](#)
 8. [Production Deployment](#)
-

Overview

AuthiChain uses **Prisma** as the ORM and migration tool for managing the PostgreSQL database schema. This guide covers all aspects of database migrations, schema management, and data seeding.

Key Features

-  **Type-safe** database access with Prisma Client
 -  **Automated migrations** for schema changes
 -  **Comprehensive seeding** scripts for development
 -  **Production-ready** migration workflows
 -  **Full transaction support** for data integrity
-

Database Schema

Core Models

User Management

- **User** - User accounts with wallet addresses and subscription tiers
- **Account** - OAuth provider accounts (NextAuth)
- **Session** - User sessions (NextAuth)
- **Subscription** - Stripe subscription management

NFT Marketplace

- **NFT** - Main NFT model with metadata and ownership
- **Collection** - NFT collections with categories
- **Auction** - Timed auctions for NFTs
- **Bid** - Auction bids and offers
- **Offer** - Direct offers on NFTs
- **NFTTransfer** - Transfer history
- **PriceHistory** - Price tracking over time

Payments & Transactions

- **Transaction** - All payment transactions (Stripe, Crypto)
- **Escrow** - Secure escrow for transactions
- **Payout** - Affiliate and creator payouts
- **MultiCurrencyPrice** - Multi-currency pricing

Marketing & Growth

- **MarketingLead** - Lead tracking
- **MarketingEvent** - Event tracking
- **MarketingCampaign** - Campaign management
- **ReferralCode** - Referral codes
- **ReferralConversion** - Referral conversions
- **AffiliatePartner** - Affiliate management

Enterprise Features

- **EnterpriseAPIKey** - API key management
- **WhiteLabelClient** - White-label clients
- **WhiteLabelIntegration** - Client integrations
- **ApiUsage** - API usage tracking

Analytics & Notifications

- **AnalyticsEvent** - User action tracking
- **Notification** - In-app notifications
- **UsageRecord** - Feature usage tracking

Additional Features

- **FavoriteNFT** - User favorites
- **WatchlistCollection** - Collection watchlist
- **LaunchBonus** - Launch bonus tracking
- **CommercialLicense** - Commercial licensing

Entity Relationship Diagram

```

User
├── Account (NextAuth)
├── Session (NextAuth)
├── Subscription
├── NFT (as creator/owner)
├── Collection (as creator)
├── Auction (as seller)
├── Bid (as bidder)
├── Transaction
├── ReferralCode
├── AffiliatePartner
├── EnterpriseAPIKey
└── Notification

NFT
├── Collection
├── Auction
├── Bid
├── Offer
├── NFTTransfer
└── PriceHistory

Transaction
└── Escrow
  
```

Quick Start

Initial Setup

```

# 1. Install dependencies (if not already done)
cd /home/ubuntu/authichain_premium/app
npm install

# 2. Set up environment variables
# Ensure DATABASE_URL is set in .env file

# 3. Run initial migration
npm run db:migrate

# 4. Seed the database with demo data
npm run db:seed
  
```

Verify Setup

```

# Open Prisma Studio to view data
npm run db:studio
  
```

Migration Commands

Development Workflow

```
# Create and apply a new migration
npm run db:migrate

# Create migration without applying
npm run db:migrate:create

# Apply pending migrations
npm run db:migrate:deploy

# Reset database (WARNING: deletes all data)
npm run db:reset

# Generate Prisma Client after schema changes
npm run db:generate

# Push schema changes without creating migrations (dev only)
npm run db:push

# Format schema file
npm run db:format
```

Individual Commands

```
# Create a named migration
npx prisma migrate dev --name add_new_feature

# Apply migrations in production
npx prisma migrate deploy

# Check migration status
npx prisma migrate status

# Resolve migration issues
npx prisma migrate resolve --applied "migration_name"
npx prisma migrate resolve --rolled-back "migration_name"

# View database in Prisma Studio
npx prisma studio
```

Seeding Data

Basic Seeding

The default seed script creates minimal test data:

```
npm run db:seed
```

Creates:

- 1 test user (john@doe.com)

- 1 subscription
- 5 sample NFTs

Comprehensive Seeding

For full development environment with all features:

```
npm run db:seed:full
```

Creates:

- 4 test users (Admin, Creator, Pro, Explorer)
- 4 subscriptions (different tiers)
- 3 NFT collections
- 5 NFTs
- 1 active auction
- 2 bids
- 3 referral codes
- 1 affiliate partner
- 2 transactions
- 1 enterprise API key
- 2 notifications
- 1 marketing lead

Test Login Credentials:

```
Admin:    admin@authichain.com / Test123!
Creator:  creator@authichain.com / Test123!
Pro:      pro@authichain.com / Test123!
Explorer: explorer@authichain.com / Test123!
```

Custom Seeding

Create custom seed scripts in `/scripts` :

```
// scripts/seed-custom.ts
import { PrismaClient } from '@prisma/client';

const prisma = new PrismaClient();

async function main() {
  // Your custom seeding logic
}

main()
  .catch((e) => {
    console.error(e);
    process.exit(1);
  })
  .finally(async () => {
    await prisma.$disconnect();
  });
```

Run with:

```
tsx scripts/seed-custom.ts
```

Schema Overview

Key Enums

Subscription Tiers

```
enum SubscriptionTier {  
  EXPLORER    // Free tier  
  CREATOR     // $49/month  
  PRO         // $149/month  
  ENTERPRISE  // $499/month  
  AGENCY      // Custom pricing  
}
```

NFT Status

```
enum NftStatus {  
  ACTIVE  
  INACTIVE  
  DELETED  
  LISTED  
  UNLISTED  
  IN_AUCTION  
  SOLD  
  BURNED  
}
```

Transaction Types

```
enum TransactionType {  
  NFT_PURCHASE  
  NFT_SALE  
  SUBSCRIPTION_PAYMENT  
  AUCTION_BID  
  AUCTION_SETTLEMENT  
  ESCROW_DEPOSIT  
  ESCROW_RELEASE  
  ROYALTY_PAYMENT  
  AFFILIATE_COMMISSION  
  REFUND  
  WITHDRAWAL  
  DEPOSIT  
}
```

Payment Methods

```
enum PaymentMethod {  
  STRIPE  
  CRYPTO_ETH  
  CRYPTO_MATIC  
  CRYPTO_USDC  
  BANK_TRANSFER  
  PAYPAL  
}
```

Important Indexes

All models include optimized indexes for:

- **Primary lookups** - Unique fields and IDs
- **Foreign key relationships** - Fast joins
- **Search queries** - Frequently filtered fields
- **Time-based queries** - CreatedAt, updatedAt timestamps
- **Status filtering** - Status enums

Troubleshooting

Common Issues

1. Migration Drift Detected

Problem: Database schema doesn't match migration history

Solution:

```
# Reset and reapply migrations (development only)  
npm run db:reset  
  
# OR manually resolve  
npx prisma migrate resolve --applied "migration_name"
```

2. Prisma Client Out of Sync

Problem: Type errors after schema changes

Solution:

```
# Regenerate Prisma Client  
npm run db:generate
```

3. Connection Issues

Problem: Cannot connect to database

Solutions:

```
# Check DATABASE_URL in .env
echo $DATABASE_URL

# Test connection
npx prisma db pull

# Verify PostgreSQL is running
psql $DATABASE_URL -c "SELECT version();"

```

4. Failed Migrations

Problem: Migration failed partially

Solution:

```
# Check migration status
npx prisma migrate status

# Mark as rolled back
npx prisma migrate resolve --rolled-back "20251019044917_initial_complete_schema"

# Try again
npm run db:migrate

```

5. Seed Failures

Problem: Seed script fails with unique constraint errors

Solution:

```
# Reset database first
npm run db:reset

# OR delete specific test data
# Use Prisma Studio to manually clean up
npm run db:studio

```

Production Deployment

Pre-Deployment Checklist

- [] All migrations tested in staging
- [] Database backup created
- [] Environment variables verified
- [] Migration rollback plan prepared
- [] Downtime window scheduled (if needed)

Deployment Steps

1. Backup Database

```
# PostgreSQL backup
pg_dump $DATABASE_URL > backup_$(date +%Y%m%d_%H%M%S).sql

```


2. Apply Migrations

```
# Set production DATABASE_URL
export DATABASE_URL="postgresql://..."

# Deploy migrations (no seed, no prompt)
npx prisma migrate deploy
```

3. Generate Client

```
# Generate Prisma Client for production
npx prisma generate
```

4. Verify Deployment

```
# Check migration status
npx prisma migrate status

# Should show: "Database schema is up to date!"
```

Rollback Procedure

If migration fails in production:

```
# 1. Restore from backup
psql $DATABASE_URL < backup_YYYYMMDD_HHMMSS.sql

# 2. Mark problematic migration as rolled back
npx prisma migrate resolve --rolled-back "migration_name"

# 3. Investigate and fix issues

# 4. Redeploy when ready
```

Zero-Downtime Migrations

For large tables or production systems:

1. **Use shadow migrations** for validation
2. **Apply backwards-compatible changes** first
3. **Deploy application code** that works with both schemas
4. **Apply breaking changes** after code deployment
5. **Monitor query performance** after migration

Best Practices

Schema Design

✅ DO:

- Use descriptive model and field names
- Add indexes for frequently queried fields
- Include `createdAt` and `updatedAt` timestamps

- Use enums for fixed value sets
- Document complex relationships with comments

✗ DON'T:

- Create circular dependencies
- Over-index (impacts write performance)
- Use generic field names like `data` or `info`
- Skip foreign key constraints

Migration Management

✓ DO:

- Create descriptive migration names
- Test migrations in development first
- Review generated SQL before applying
- Keep migrations small and focused
- Document breaking changes

✗ DON'T:

- Edit applied migrations
- Skip migrations in production
- Delete migration files
- Mix schema and data changes

Data Seeding

✓ DO:

- Use upsert for idempotent seeds
- Create separate seeds for different environments
- Include realistic test data
- Document seed data credentials
- Clean up test data in production

✗ DON'T:

- Seed production databases
 - Use weak passwords (even for test users)
 - Create excessive test data
 - Hard-code production secrets
-

Useful Commands Reference

```
# === Migration Commands ===
npm run db:migrate          # Create and apply migration
npm run db:migrate:create   # Create only (no apply)
npm run db:migrate:deploy   # Apply migrations (production)
npm run db:reset            # Reset database (dev only)

# === Schema Commands ===
npm run db:generate         # Generate Prisma Client
npm run db:push             # Push schema (no migration)
npm run db:format          # Format schema file
npm run db:studio          # Open Prisma Studio

# === Seeding Commands ===
npm run db:seed             # Basic seed
npm run db:seed:full        # Comprehensive seed

# === Prisma CLI Direct ===
npx prisma migrate dev
npx prisma migrate deploy
npx prisma migrate status
npx prisma studio
npx prisma db pull
npx prisma db push
npx prisma generate
npx prisma format
```

Additional Resources

- [Prisma Documentation](https://www.prisma.io/docs) (https://www.prisma.io/docs)
- [Prisma Migrate Guide](https://www.prisma.io/docs/concepts/components/prisma-migrate) (https://www.prisma.io/docs/concepts/components/prisma-migrate)
- [PostgreSQL Documentation](https://www.postgresql.org/docs/) (https://www.postgresql.org/docs/)
- [AuthiChain API Documentation](#) (./API_DOCUMENTATION.md)
- [AuthiChain Setup Guide](#) (./SETUP_GUIDE.md)

Support

For issues or questions:

- Check [Troubleshooting](#) section
- Review Prisma documentation
- Contact development team

Last Updated: October 19, 2025

Maintained by: AuthiChain Development Team